

10 Open-Source AI Tools, One Router, Zero Glue Code

10 Open-Source AI Tools, One Router, Zero Glue Code: How I Built a Serverless RAG Workbench on Nebius

AI Engineering · June 23, 2026

Submission for the #NebiusServerlessChallenge

I was tired of stitching together crawlers, vector databases, LLM endpoints, PDF tools, and chat UIs every time I started a new AI project. So I built one workbench that connects all of them — and runs entirely serverless on Nebius AI Cloud.

A few months ago, I caught myself doing the same dance for the fifth time.

Spin up a crawler. Chunk the text. Push embeddings into a vector store. Wire a FastAPI layer to an LLM endpoint. Then remember that someone will eventually ask, “Can it read PDFs too?” So I added Stirling PDF. Then, “Can we chat with it?” So I added Open WebUI. Then, “Can it scrape a website visually?” So I added Browser Use and Maxun. Every project became a pile of shell scripts, broken networking, and duplicated Docker Compose files.

That’s the dirty secret of the current AI boom: the models are amazing, but the integration layer is still mostly manual labor. We don’t have a shortage of great open-source tools. We have a shortage of clean ways to make them work together.

For the **Nebius Serverless AI Builders Challenge**, I decided to stop doing the integration dance and just build the dance floor once. The result is the **Nebius 10-in-1 AI Workbench**: a single FastAPI router that exposes ten open-source AI tools as one coherent pipeline, deployed serverless on Nebius.

- **GitHub repo:** github.com/pyscalp/10in1-ai
- **Live router:** <http://195.242.13.3:8000>

The idea: one HTTP router to rule them all

Instead of making users talk to ten different ports, ten different APIs, and ten different documentation pages, the workbench puts a thin router in front of everything. The router knows how to:

- Ingest a URL, crawl it, chunk it, embed it, and store the vectors.
- Answer a question using retrieved context plus a Nebius Serverless LLM endpoint.
- Reach PDF tools, a chat UI, browser agents, and visual scrapers through

the same internal network.

A user — or another application — only needs one URL and one Bearer token.

The ten tools in the stack are:

#	Tool	Role
1	Crawl4AI	Async, LLM-ready web crawling
2	Stirling PDF	Self-hosted PDF conversion and OCR
3	Open WebUI	ChatGPT-style self-hosted chat interface
4	Supabase / PostgreSQL + pgvector	Vector storage and metadata
5	Langflow	Visual builder for RAG and agent flows
6	Dify	Ship AI apps with workflows and RAG
7	Browser Use	AI browser agent for forms and automation
8	Maxun	Point-and-click visual scraping workflows
9	OpenHands	Autonomous coding agent for pipeline patches
10	Coolify	Self-hosting PaaS for router and frontends

The first four are already wired into the live RAG flow. The rest are containerized and reachable; the adapters to expose them through the router are the next layer on the roadmap.

How the live RAG flow works

Ingestion and question answering are the two endpoints I use most often. Here is the entire demo flow in plain English:

1. Send a URL to `POST /pipeline/ingest`.
2. Crawl4AI fetches the page and turns it into clean markdown.
3. The router chunks the text and embeds it with `sentence-transformers/all-MiniLM-L6-v2`.
4. Chunks and embeddings go into a managed PostgreSQL + pgvector database on Nebius.
5. Send a question to `POST /pipeline/ask`.
6. The router retrieves the top-k chunks and forwards them to a Nebius Serverless LLM endpoint.
7. The LLM returns an answer grounded in the retrieved context.

Everything behind the router uses Nebius internal networking. Only the router itself is exposed publicly, which keeps the attack surface small.

If you want to try it, the public router is running now:

```
export ROUTER_IP=195.242.13.3:8000
```

```

export ROUTER_TOKEN=<your-router-token>

# Health check
curl -H "Authorization: Bearer ${ROUTER_TOKEN}"
http://${ROUTER_IP}/health

# Ingest a public URL
curl -X POST http://${ROUTER_IP}/pipeline/ingest \
  -H "Authorization: Bearer ${ROUTER_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{"url":"https://docs.nebius.com/llms.txt","source":"nebius-llms"}'

# Ask a RAG question
curl -X POST http://${ROUTER_IP}/pipeline/ask \
  -H "Authorization: Bearer ${ROUTER_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{"question":"What AI services does Nebius offer?","top_k":2}'

```

Replace `$ROUTER_TOKEN` with your own token, or open an issue in the repo if you want temp access for testing. The commands intentionally use a placeholder so I don't accidentally leak credentials.

Why Nebius fit this project

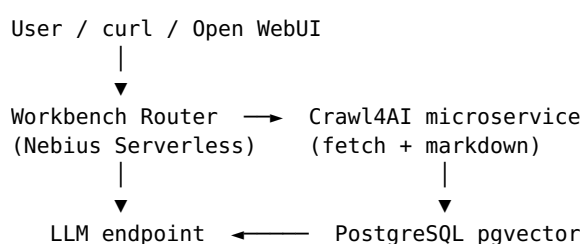
I could have run this on a rented GPU server, but that would have defeated the point. The workbench is designed to feel serverless: endpoints start when you need them, scale without my intervention, and stop costing money when you stop calling them.

Nebius made that possible in a few specific ways:

- **Serverless AI Endpoints** — the router, Crawl4AI, and LLM run as managed endpoints. I don't operate Kubernetes or babysit long-running instances.
- **Managed PostgreSQL with pgvector** — vectors and metadata live in a service I don't have to patch or back up myself.
- **Integrated Container Registry** — images are built locally and pushed straight to Nebius, which keeps the deploy loop tight.
- **Cost control** — endpoints can be paused or deleted between demos, and CPU presets keep the initial stack cheap. When I want a larger model, I can swap in a GPU-backed vLLM endpoint without rewriting the router.

That last point matters more than it sounds. A lot of cloud providers make the first deployment free but the second iteration expensive. With Nebius, I could iterate on the router code, push a new image, and redeploy in a few minutes.

Architecture at a glance



(Nebius Serverless) (chunks + embeddings)

The router is the only public entry point. Crawl4AI and the LLM are Nebius Serverless Endpoints. PostgreSQL is a managed Nebius database. Internal traffic never leaves the Nebius network, so I avoid the classic problem of exposing ten different microservices to the open internet.

A full Mermaid diagram is in the repository README if you prefer a visual version.

What I learned along the way

The biggest surprise wasn't technical. It was that most of the work had nothing to do with AI and everything to do with plumbing.

For example, Crawl4AI is great at producing clean markdown, but it doesn't care how you embed it. pgvector is great at vector search, but it doesn't care how you chunk. Open WebUI is great as a chat interface, but it expects a specific API shape. The actual value of this workbench is the thin layer that translates between those assumptions.

A few concrete lessons:

- **Chunking is underrated.** The same content performs very differently depending on chunk size and overlap. I settled on a configurable chunker rather than a hard-coded value.
 - **Internal networking matters.** Keeping tools on the same VPC or internal network removed a huge amount of auth and TLS pain. Public IPs are only for the router.
 - **Logs are evidence.** Nebius wants proof the submission runs on their platform, so I wrote the router to return clear ingestion logs and timing headers. Those logs become the screenshots and benchmarks for the submission form.
 - **Don't commit secrets.** This sounds obvious, but the first draft of the docs contained internal IP addresses that leaked from local environment files. I had to rewrite Git history to remove them. Now everything sensitive is loaded from environment variables.
-

What comes next

The workbench is already usable as a RAG pipeline, but I am not done yet.

The next steps are:

- Complete router adapters for Langflow, Dify, Browser Use, Maxun, OpenHands, and Coolify so they become first-class endpoints.
- Add a Nebius Serverless AI Job that batch-crawls and embeds an entire website, not just one URL.
- Upgrade the current CPU LLM endpoint to a GPU-backed vLLM endpoint once quota allows, which will unlock larger models and longer context windows.

The pattern, though, already works: one router, many open-source tools, all running serverless.

Final thought

If you are building AI prototypes, you already know the feeling. Every new project starts with the same scaffolding: crawl, embed, retrieve, generate, chat, export, repeat. The Nebius 10-in-1 AI Workbench removes that scaffolding and lets you start with the actual problem.

It is not a magic platform. It is just a clean integration layer on top of tools that already do the hard work. And because it runs on Nebius Serverless, you only pay for the compute you actually use while iterating.

If you want to see the code, the live demo, or just tell me why this is a terrible idea, everything is public:

- **Project repo:** github.com/pyscalp/10in1-ai
- **Live endpoint:** <http://195.242.13.3:8000>
- **Challenge:** #NebiusServerlessChallenge

*Tags: #NebiusServerlessChallenge #RAG #AIWorkbench #OpenSource
#NebiusAI #Crawl4AI #pgvector #vLLM*